Predicting Gene-Pair Responses Beyond Additivity with Biological Embeddings and Set Attention

Pramika Garg | UNI: pg2899 | Interim report

Introduction

Predicting how gene targets act together could help prioritize combinations for therapy. Joint activation can reinforce or suppress responses and alter differentiation [5]. I ask whether biological gene features and attention improve predictions for unseen combinations. The main comparison is additivity: adding measured single-gene effects. Ahlmann-Eltze et al. found this approach outperformed the deep-learning methods they tested [1]. I compare it with GEARS [6] and an extension using DepMap features and set attention (Fig. 1). These K562 leukemia-cell experiments can inform target prioritization, but cannot establish drug efficacy.

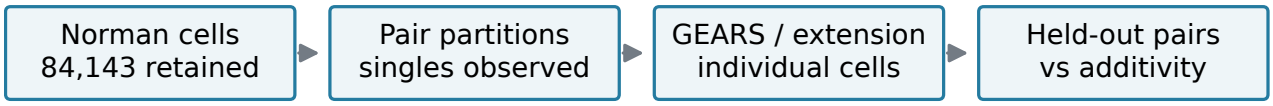


Figure 1: Completed workflow. Baselines use five random pair partitions. Both cell-level models share one partition, one initialization and five training epochs.

Data and exploratory analysis

I downloaded the scFoundation-processed Norman Perturb-seq release used in [1]: 84,986 cells, 19,264 genes and 227 conditions. I removed 843 cells from two conditions involving *IER5L*, absent from the supplied Gene Ontology (GO) dictionary. This left **84,143 cells, 100 single-gene conditions, 124 pairs and one control condition**. The control contains 7,275 cells. Conditions contain 46–7,275 cells, with median 281 (Fig. 2A).

I processed the sparse matrix in chunks to calculate condition means, variances and cell summaries. Values were nonnegative and continuous, with `log1p` metadata. I retained this scale without normalizing again. No stored values were nonfinite; 85.1% of entries were zero (Fig. 2B). Principal component analysis (PCA) of condition means for the 1,000 highest-mean control genes explains 35.1% and 22.6% of variance in its first two components. Single-gene and pair conditions overlap (Fig. 2C). Hemoglobin genes *HBZ*, *HBG2* and *HBG1* have the largest absolute PC2 loadings. This exploratory PCA was not model input.

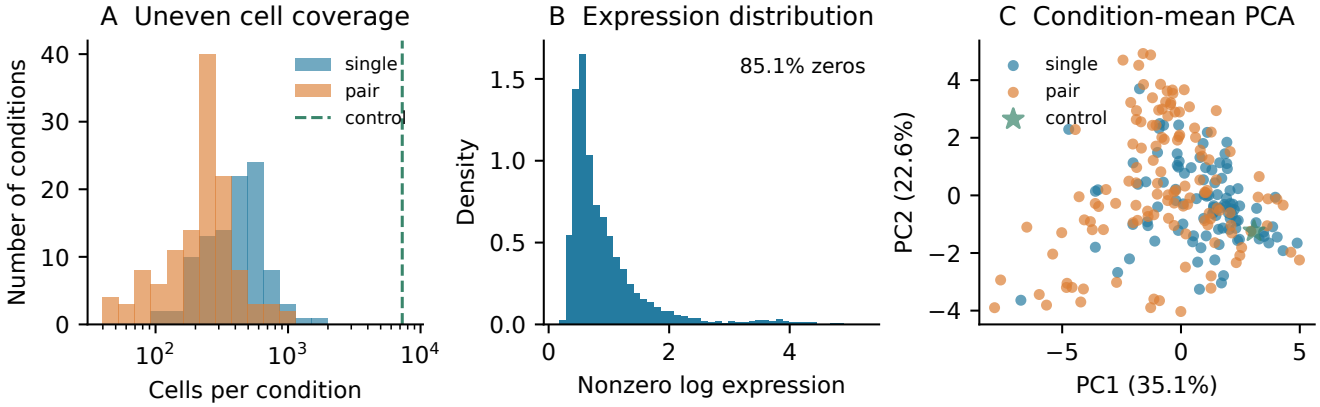


Figure 2: Exploratory analysis. A: cell counts per condition; the dashed line marks the control count. B: sampled nonzero expression values; the zero fraction uses the full retained matrix. C: centered, unscaled PCA of all 225 condition means, colored by condition type.

Methods

Withholding gene pairs. I created five random partitions using seeds 101, 202, 303, 404 and 505. Each assigns 62 pairs to training, 31 to validation and 31 to testing. All single-gene and control conditions remain available for training. Cells from each condition stay together, so a test pair is always unseen as a combination. The five partitions give 155 test appearances covering 98 distinct pairs.

Baselines and evaluation. The additive prediction is $\widehat{\mu}^{\text{add}}_{AB} = \mu_A + \mu_B - \mu_{\text{ctrl}}$, where μ denotes mean expression in a condition. Subtracting the control once avoids counting it twice. Two simpler baselines predict the control mean or the equally weighted mean of training perturbation conditions. I measured mean squared error (MSE) on the same 1,000 control-selected genes. Lower MSE means better prediction.

For each pair p in partition s, I calculated $D_{p,s} = \text{MSE}_{\text{add},p,s} - \text{MSE}_{\text{model},p,s}$. Positive values favor the model. I averaged repeated test results within pairs, then gave each pair equal weight. Intervals use 10,000 bootstrap resamples of pairs. They describe uncertainty across pairs for these fitted predictors. They do not capture variation from retraining, and overlapping partitions or shared genes do not provide independent biological replication.

Neural models. I trained the pinned upstream GEARS implementation and an extension motivated by MORPH [3]. The extension uses fixed DepMap profiles of gene dependencies across cancer cell lines. It replaces GEARS' sum of gene representations with set attention [4]. Attention learns how to combine the representations without depending on their order. I used the labeled DepMap 22Q4 release after excluding K562 and its derivative. Features cover 96 of 100 targets. Missing targets receive zero vectors and a presence indicator. A learned projection adds normalized features after the GO encoder; self-attention and learned-seed pooling then combine the target representations. Both models predict all 19,264 genes, use 64 hidden units and the official GO graph of 9,853 nodes. Their coexpression graph uses only individual training cells from single-gene and control conditions.

Both models trained on 67,592 cells. The GEARS loss penalizes the fourth power of expression error and errors in response direction. I used Adam with learning rate 10^{-3}, weight decay 5×10^{-4}, batches of 32 and learning-rate halving each epoch. T4 timing allowed five epochs per model within the shared three-hour budget. Both used partition 101, initialization seed 17 and FP16 precision. Checkpoints were selected using validation data only.

To predict condition means, I used the mean control expression as input and evaluated one condition at a time. At batch size one, this exactly averages predictions over individual control inputs because the input expression enters through a final additive path. I retained upstream graph batching, which can make predictions depend on position within a training batch. This behavior, the processed dataset, short schedule and validation metric limit comparison with published GEARS results.

Preliminary results

Additivity was the strongest baseline. Across the 98 distinct test pairs, mean MSE was **0.00929 for additivity**, 0.0206 for the training mean and 0.0364 for the control mean (Fig. 3A). Additivity also had lower mean error in each of the five partitions. The training-mean predictor's paired difference was -0.0113, with a 95% interval of $[-0.0139, -0.00872]$.

On the same 31 test pairs in partition 101, MSE was **0.01098 for additivity**, 0.01375 for GEARS and 0.01985 for the extension (Fig. 3B). Both models completed five epochs, and validation selected epoch 3 for each. GEARS beat additivity on 7 of 31 pairs; the extension did so on 6. Their mean paired differences were -0.00277 and -0.00886. Thus, neither improved on additivity in this run. Because the extension changes both features and aggregation, these results cannot identify either component's separate effect. They also cover only one partition and initialization. Total timed execution was 109 minutes.

I assessed repeatability of responses beyond additivity by independently dividing control, single-gene and pair cells into halves. Across 124 pairs, the median correlation between interaction residuals from the two halves was 0.435. This measures within-screen repeatability, rather than independent biological replication.

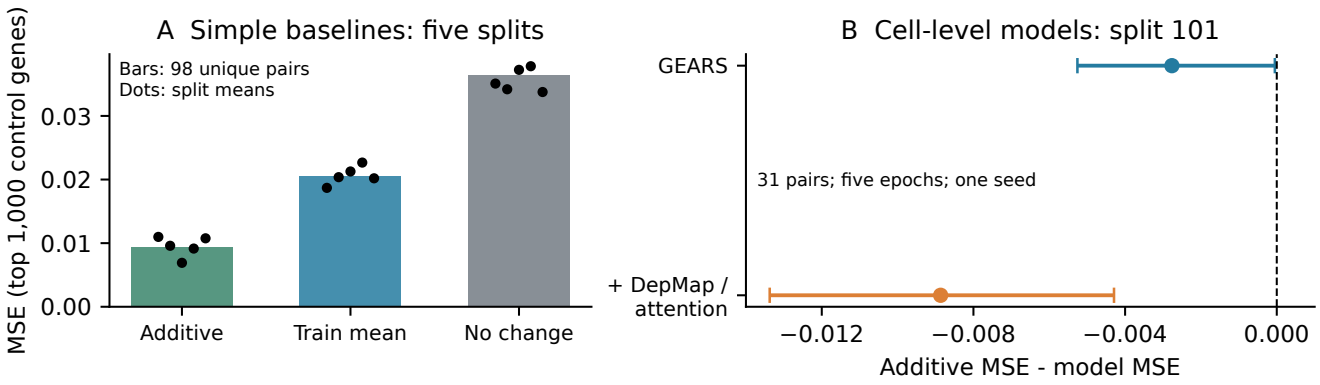


Figure 3: Prediction performance. A: bars give equal weight to each distinct pair after averaging its test appearances; dots show five partition means. B: additive-minus-model MSE on the same 31 pairs, with 95% pair-bootstrap intervals. Negative values favor additivity. These intervals do not measure variability across partitions or training initializations.

Next steps

I will repeat training across five partitions and two initializations, assess longer schedules, and separate the effects of gene features and aggregation in the six planned configurations. GenePT [2] has so far been tested only in earlier models trained on condition means. I will also examine unstable validation errors and the upstream batch-dependent graph implementation. Few combinations, four missing DepMap targets and noisy interaction residuals may make improvements difficult to detect. The prespecified final comparison remains DepMap with attention against additivity, with model selection restricted to validation data. The current results do not establish that embeddings cannot help or that interactions are absent.

Reproducibility. The accompanying `interim/` and `gpu/` folders contain scripts, partitions, results, dependency versions and source hashes. I independently recalculated errors from exported predictions for every reported scoring subset and checked file hashes, checkpoint selection and elapsed time. Earlier pilots are preserved. The small code package excludes public raw data and large checkpoints.

References

[1] C. Ahlmann-Eltze, W. Huber, and S. Anders. Deep-learning-based gene perturbation effect prediction does not yet outperform simple linear baselines. *Nature Methods*, 22:1657–1661, 2025. doi: 10.1038/s41592-025-02772-6. Code and data.

[2] Y. Chen and J. Zou. Simple and effective embedding model for single-cell biology built from ChatGPT. *Nature Biomedical Engineering*, 9:483–493, 2025. doi: 10.1038/s41551-024-01284-6.

[3] C. He, J. Zhang, M. A. Dahleh, and C. Uhler. MORPH predicts the single-cell outcome of genetic perturbations across conditions and data modalities. *bioRxiv*, 2025. doi: 10.1101/2025.06.27.661992. v2; prior comparison also available in v1.

[4] J. Lee et al. Set Transformer: A framework for attention-based permutation-invariant neural networks. In *Proceedings of ICML*, volume 97 of *PMLR*, pages 3744–3753, 2019. URL `https://proceedings.mlr.press/v97/lee19d.html`.

[5] T. M. Norman et al. Exploring genetic interaction manifolds constructed from rich single-cell phenotypes. *Science*, 365:786–793, 2019. doi: 10.1126/science.aax4438.

[6] Y. Roohani, K. Huang, and J. Leskovec. Predicting transcriptional outcomes of novel multigene perturbations with GEARS. *Nature Biotechnology*, 42:927–935, 2024. doi: 10.1038/s41587-023-01905-6.